



PROCESO DE DESARROLLO DE SOFTWARE

Grupo 13

Trabajo Práctico Obligatorio

Integrantes:

Tomas Szkarlatiuk, 1190913

Delia Tomas, 1188972

Rocha Germán, 1195326

Informe final

Sistema de Gestión Hotelera y Servicios para Huéspedes

Profesora:

LEON MARIA ANGELA

1. Introducción	4
2. Descripción del problema	5
3. Objetivos del sistema	5
Objetivo general	5
Objetivos específicos	5
4. Requisitos funcionales	6
5. Requisitos no funcionales	6
6. Casos de uso	7
Casos de uso implementados	7
Diagrama de Casos de Uso	8
7. Diagrama de clases	8
Diagrama de Clases UML	8
8. Correspondencia entre UML e implementación	8
9. Justificación del diseño arquitectónico	9
Dominio	9
Controladores	9
Persistencia	9
Fachada	10
10. Patrones de diseño GoF aplicados	10
10.1. Factory Method	10
Problemática detectada	10
Solución con el patrón	10
Ventaja obtenida	10
10.2. Singleton	11
Problemática detectada	11
Solución con el patrón	11
Ventaja obtenida	11
10.3. Decorator	11
Problemática detectada	11
Solución con el patrón	11
Ventaja obtenida	11
10.4. Facade	12
Problemática detectada	12
Solución con el patrón	12
Ventaja obtenida	12
10.5. State	12
Problemática detectada	12

Solución con el patrón	12
Ventaja obtenida	12
10.6. Strategy	12
Problemática detectada	12
Solución con el patrón	13
Ventaja obtenida	13
11. Principios SOLID aplicados	13
SRP – Single Responsibility Principle	13
Aplicación en el sistema	13
Ejemplos	13
OCP – Open/Closed Principle	14
Aplicación en el sistema	14
Ejemplos	14
DIP – Dependency Inversion Principle	14
Aplicación en el sistema	14
Ejemplos	14
12. Patrones GRASP aplicados	14
Experto en Información	15
Aplicación	15
Relación con SOLID	15
Creador	15
Aplicación	15
Relación con SOLID	15
Controlador	15
Aplicación	15
Relación con SOLID	16
Bajo Acoplamiento	16
Aplicación	16
Relación con SOLID	16
Alta Cohesión	16
Aplicación	16
Relación con SOLID	16
13. Instrucciones de despliegue	17
14. Evidencia de ejecución	17
Casos de prueba ejecutados	17
Evidencia 1 – Registro de huéspedes	18
Evidencia 2 – Registro de habitaciones	18
Evidencia 3 – Creación de reserva	18
Evidencia 4 – Confirmación de reserva	18
Evidencia 5 – Aplicación de descuentos y servicios	18
Evidencia 6 – Registro de pago	18

Evidencia 7 – Historial de reservas	18
Evidencia 8 – Validación de transición inválida	18
15. Historial de versiones	19
16. Conclusión	19

INFORME FINAL

Sistema de Gestión Hotelera y Servicios para Huéspedes

1. Introducción

El presente informe corresponde a la entrega final del Trabajo Práctico Obligatorio de la materia Proceso de Desarrollo de Software. El proyecto desarrollado consiste en un Sistema de Gestión Hotelera implementado en Java utilizando programación orientada a objetos.

El sistema permite administrar huéspedes, habitaciones, reservas, pagos, promociones y servicios adicionales asociados a una estadía. Para su desarrollo se aplicaron patrones de diseño GoF, principios SOLID y patrones GRASP, con el objetivo de construir una solución organizada, extensible y de fácil mantenimiento.

Durante las distintas etapas del trabajo se realizó el análisis del problema, el diseño mediante diagramas UML y la implementación completa del sistema, manteniendo coherencia entre el diseño propuesto y el código desarrollado.

2. Descripción del problema

La gestión de un hotel involucra múltiples procesos relacionados con huéspedes, habitaciones, reservas, promociones y pagos. Cuando estas actividades se realizan de forma manual o mediante herramientas poco integradas, pueden producirse errores en la disponibilidad de habitaciones, inconsistencias en la información registrada y dificultades para administrar los distintos servicios ofrecidos por el establecimiento.

Para resolver esta problemática se desarrolló un Sistema de Gestión Hotelera capaz de centralizar la administración de los principales procesos operativos. La solución permite registrar huéspedes, gestionar habitaciones, crear reservas, administrar sus estados, aplicar promociones, agregar servicios adicionales y registrar pagos, proporcionando una estructura organizada y preparada para futuras ampliaciones.

3. Objetivos del sistema

Objetivo general

Desarrollar un Sistema de Gestión Hotelera que permita administrar huéspedes, habitaciones, reservas, promociones, servicios adicionales y pagos mediante una solución orientada a objetos implementada en Java.

Objetivos específicos

- Registrar huéspedes y habitaciones.
 - Consultar disponibilidad de habitaciones.
 - Crear reservas asociadas a huéspedes y habitaciones.
 - Confirmar, cancelar y finalizar reservas.
 - Aplicar promociones y descuentos.
 - Incorporar servicios adicionales a las reservas.
 - Registrar pagos asociados a una estadia.
 - Aplicar patrones de diseño, principios SOLID y patrones GRASP.
 - Mantener una arquitectura organizada y extensible.
-

4. Requisitos funcionales

El sistema implementa los siguientes requerimientos funcionales:

Código	Requisito Funcional
RF1	Registrar huéspedes
RF2	Registrar habitaciones
RF3	Consultar habitaciones disponibles
RF4	Crear reservas
RF5	Confirmar reservas
RF6	Cancelar reservas
RF7	Finalizar reservas
RF8	Aplicar promociones y descuentos
RF9	Agregar servicios adicionales a una reserva
RF10	Calcular el costo total de una reserva

RF11	Registrar pagos asociados a reservas
RF12	Consultar historial de reservas
RF13	Notificar eventos relacionados con una reserva

Los requerimientos definidos durante las etapas previas fueron implementados y posteriormente validados mediante los escenarios de prueba realizados durante la fase final del proyecto.

5. Requisitos no funcionales

El sistema fue desarrollado considerando los siguientes requisitos no funcionales:

Código	Requisito No Funcional
RNF1	Implementación en Java
RNF2	Uso de programación orientada a objetos
RNF3	Organización clara en paquetes
RNF4	Bajo acoplamiento y alta cohesión
RNF5	Aplicación de patrones de diseño
RNF6	Aplicación de principios SOLID y patrones GRASP
RNF7	Coherencia entre UML e implementación
RNF8	Arquitectura preparada para futuras ampliaciones
RNF9	Reutilización de componentes
RNF10	Ejecución funcional de los principales casos de uso

La arquitectura desarrollada permite incorporar nuevas funcionalidades sin modificar significativamente la estructura existente, favoreciendo la mantenibilidad y evolución futura del sistema.

6. Casos de uso

El Sistema de Gestión Hotelera implementa los principales procesos necesarios para la administración de huéspedes, habitaciones, reservas y pagos.

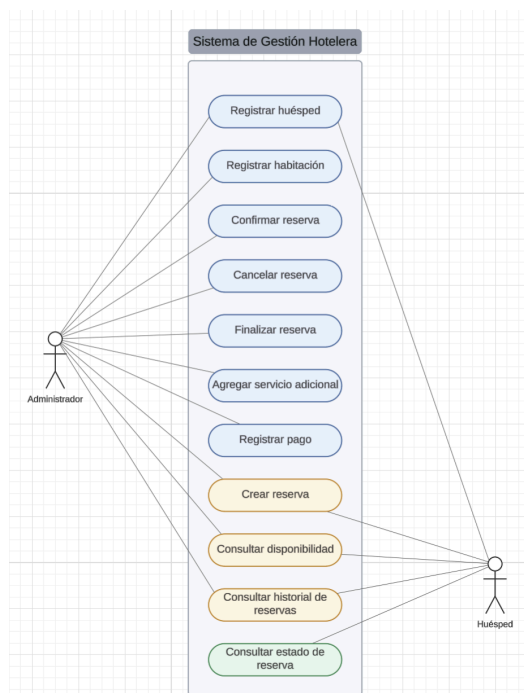
Los casos de uso desarrollados permiten cubrir las funcionalidades definidas durante el análisis del problema y representan las operaciones principales realizadas por los usuarios del sistema.

Casos de uso implementados

- Registrar huésped.
- Registrar habitación.
- Consultar disponibilidad de habitaciones.
- Crear reserva.
- Confirmar reserva.
- Cancelar reserva.
- Finalizar reserva.
- Aplicar descuentos y promociones.
- Agregar servicios adicionales.
- Registrar pago.
- Consultar historial de reservas.

Diagrama de Casos de Uso

El siguiente diagrama representa las funcionalidades principales implementadas y los actores que interactúan con el sistema.



Durante la implementación final se incorporó una diferenciación básica de roles dentro de la interfaz gráfica. El Administrador dispone de acceso a todas las funcionalidades de gestión del hotel, mientras que el huésped puede consultar disponibilidad, realizar reservas y consultar el estado e historial de sus reservas.

7. Diagrama de clases

El diseño del sistema fue modelado mediante un diagrama de clases UML que representa las entidades principales del dominio hotelero, las relaciones entre ellas y los patrones de diseño implementados.

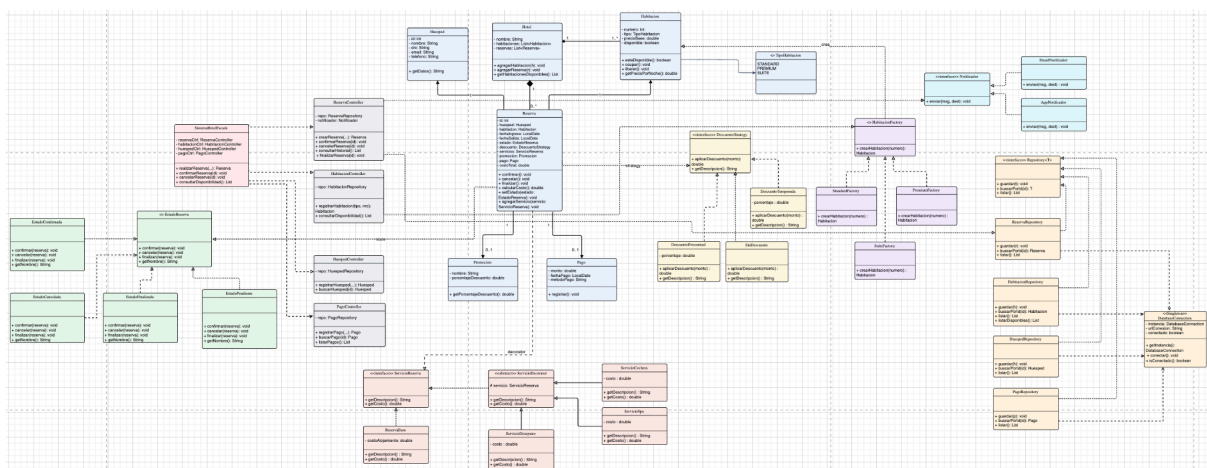
Entre las clases principales se encuentran:

- Hotel
- Habitación
- Huésped
- Reserva
- Pago
- Promoción

Asimismo, el modelo incorpora los distintos controladores, repositorios y clases asociadas a los patrones de diseño utilizados durante la implementación.

Las relaciones representadas en el diagrama mantienen correspondencia directa con el código desarrollado, permitiendo reflejar de manera precisa la estructura final del sistema.

Diagrama de Clases UML



8. Correspondencia entre UML e implementación

Los diagramas UML fueron actualizados durante la fase final del proyecto para reflejar fielmente la implementación realizada en Java.

El modelo incorpora las clases de dominio, controladores, repositorios y patrones de diseño efectivamente implementados, manteniendo coherencia entre el análisis realizado, el diseño propuesto y la solución final desarrollada.

Las relaciones de asociación, composición, herencia y dependencia representadas en los diagramas corresponden a las utilizadas en el código fuente del proyecto.

De esta forma se garantiza la trazabilidad entre los requerimientos definidos inicialmente y la implementación final entregada.

9. Justificación del diseño arquitectónico

El sistema fue desarrollado utilizando una arquitectura organizada en paquetes con responsabilidades claramente diferenciadas. Esta estructura permite mantener el código ordenado, facilitar su mantenimiento y reducir el acoplamiento entre componentes.

Dominio

Contiene las entidades principales del negocio:

- Hotel
- Habitacion
- Huesped
- Reserva
- Pago
- Promocion
- TipoHabitacion

Estas clases representan la información y comportamiento asociado a la gestión hotelera.

Controladores

Los casos de uso son coordinados mediante:

- HuespedController
- HabitacionController
- ReservaController
- PagoController

Su responsabilidad es actuar como intermediarios entre la interfaz, el dominio y la persistencia.

Persistencia

La capa de persistencia está compuesta por:

- Repository
- HuespedRepository
- HabitacionRepository
- ReservaRepository
- PagoRepository
- DatabaseConnection

Esta capa desacopla el almacenamiento de datos de la lógica de negocio.

Fachada

La clase SistemaHotelFacade actúa como punto único de acceso al sistema, simplificando la interacción con los distintos subsistemas y reduciendo el acoplamiento entre componentes.

La combinación de estas capas permite mantener una arquitectura modular, organizada y preparada para futuras ampliaciones.

10. Patrones de diseño GoF aplicados

Durante el desarrollo se implementaron patrones de diseño pertenecientes a las categorías creacional, estructural y de comportamiento para resolver distintos problemas presentes en el dominio de negocio.

10.1. Factory Method

Problemática detectada

El sistema necesita crear habitaciones de distintos tipos (Standard, Premium y Suite) sin que los controladores dependan directamente de clases concretas.

Solución con el patrón

Se implementó la clase abstracta HabitacionFactory y las fábricas concretas:

- StandardFactory
- PremiumFactory
- SuiteFactory

Cada fábrica encapsula la lógica de creación correspondiente a un tipo específico de habitación.

Ventaja obtenida

Permite agregar nuevos tipos de habitación sin modificar la lógica existente, favoreciendo la extensibilidad y reduciendo el acoplamiento.

10.2. Singleton

Problemática detectada

Se requiere un único punto de acceso para la configuración y gestión de la persistencia del sistema.

Solución con el patrón

La clase DatabaseConnection implementa Singleton mediante una instancia única compartida por toda la aplicación.

Ventaja obtenida

Evita la duplicación de recursos y centraliza el acceso a la información almacenada.

10.3. Decorator

Problemática detectada

Una reserva puede incluir distintos servicios adicionales, como desayuno, spa o cochera. Utilizar herencia para representar todas las combinaciones posibles generaría una gran cantidad de clases.

Solución con el patrón

Se implementaron:

- ServicioReserva
- ServicioDecorator
- ServicioDesayuno
- ServicioSpa
- ServicioCochera

Cada decorador agrega funcionalidad adicional a una reserva sin modificar la estructura base.

Ventaja obtenida

Permite incorporar nuevos servicios de forma dinámica y flexible.

10.4. Facade

Problemática detectada

El sistema posee múltiples controladores y componentes internos. Exponerlos directamente aumentaría la complejidad para los clientes del sistema.

Solución con el patrón

Se implementó la clase SistemaHotelFacade como punto único de acceso a las operaciones principales.

Ventaja obtenida

Simplifica el uso del sistema y reduce el acoplamiento entre componentes.

10.5. State

Problemática detectada

Las reservas atraviesan distintos estados y no todas las transiciones son válidas.

Solución con el patrón

Se implementa la interfaz EstadoReserva y los estados concretos:

- EstadoPendiente
- EstadoConfirmada
- EstadoCancelada
- EstadoFinalizada

La clase Reserva delega el comportamiento correspondiente al estado actual.

Ventaja obtenida

Permite encapsular las reglas de transición y evitar estructuras condicionales complejas.

10.6. Strategy

Problemática detectada

El cálculo de descuentos puede variar según promociones o temporadas.

Solución con el patrón

Se implementa la interfaz DescuentoStrategy junto con:

- SinDescuento
- DescuentoPorcentual
- DescuentoTemporada

La clase Reserva utiliza una estrategia concreta para calcular el descuento correspondiente.

Ventaja obtenida

Permite incorporar nuevas políticas de descuento sin modificar la lógica principal del sistema.

11. Principios SOLID aplicados

Durante el desarrollo del sistema se aplicaron distintos principios SOLID con el objetivo de obtener una arquitectura más mantenible, extensible y desacoplada.

SRP – Single Responsibility Principle

Este principio establece que una clase debe tener una única responsabilidad.

Aplicación en el sistema

La arquitectura se encuentra dividida en paquetes con responsabilidades específicas:

- Las clases del dominio representan entidades del negocio.
- Los controladores coordinan los casos de uso.
- Los repositorios administran la persistencia.
- La fachada centraliza el acceso al sistema.

Ejemplos

- Reserva administra la información y comportamiento de una reserva.
- ReservaRepository administra el almacenamiento de reservas.
- ReservaController coordina las operaciones relacionadas con reservas.
- SistemaHotelFacade simplifica el acceso a las funcionalidades principales.

OCP – Open/Closed Principle

Este principio establece que las clases deben estar abiertas a la extensión y cerradas a la modificación.

Aplicación en el sistema

El sistema permite incorporar nuevas funcionalidades sin modificar el código existente.

Ejemplos

- Nuevos tipos de habitación mediante Factory Method.
- Nuevas promociones mediante Strategy.
- Nuevos estados mediante State.
- Nuevos servicios mediante Decorator.

Esto permite ampliar el sistema manteniendo estable la lógica ya implementada.

DIP – Dependency Inversion Principle

Este principio establece que los módulos de alto nivel deben depender de abstracciones y no de implementaciones concretas.

Aplicación en el sistema

El proyecto utiliza distintas interfaces para desacoplar componentes.

Ejemplos

- DescuentoStrategy
- EstadoReserva
- Notificador
- Repository

Gracias a estas abstracciones, es posible reemplazar implementaciones concretas sin afectar el resto del sistema.

12. Patrones GRASP aplicados

Los patrones GRASP fueron utilizados para distribuir responsabilidades de manera adecuada dentro del sistema y mantener una arquitectura organizada.

Experto en Información

La responsabilidad se asigna a la clase que posee la información necesaria para realizar una tarea.

Aplicación

- Reserva calcula el costo total de una estadía.
- Habitación conoce su precio y disponibilidad.
- Pago administra la información asociada a un pago.

Relación con SOLID

Se vincula principalmente con SRP, ya que cada clase administra información relacionada con su propia responsabilidad.

Creador

La creación de objetos se asigna a quien los contiene o utiliza estrechamente.

Aplicación

- HabitaciónController utiliza las fábricas para crear habitaciones.
- ReservaController coordina la creación de reservas.

Relación con SOLID

Se relaciona con OCP, ya que la creación puede extenderse mediante nuevas fábricas sin modificar las existentes.

Controlador

Los casos de uso son gestionados mediante clases especializadas.

Aplicación

- HuespedController

- HabitaciónController
- ReservaController
- PagoController

Estas clases coordinan las operaciones del sistema sin cargar de responsabilidades a las entidades del dominio.

Relación con SOLID

Se relaciona con SRP al separar la lógica de coordinación de la lógica de negocio.

Bajo Acoplamiento

Busca minimizar las dependencias entre componentes.

Aplicación

- SistemaHotelFacade reduce dependencias directas.
- Uso de interfaces como EstadoReserva y DescuentoStrategy.
- Separación entre dominio, controladores y persistencia.

Relación con SOLID

Se vincula principalmente con DIP, ya que las dependencias se establecen sobre abstracciones.

Alta Cohesión

Busca que cada clase concentre responsabilidades relacionadas entre sí.

Aplicación

- Reserva administra reservas.
- Pago administra pagos.
- Los repositorios administran persistencia.
- Los controladores coordinan casos de uso.

Relación con SOLID

Se relaciona con SRP, ya que cada clase mantiene una responsabilidad claramente definida.

La aplicación conjunta de principios SOLID y patrones GRASP permitió desarrollar una solución organizada, extensible y coherente con el diseño UML y la implementación final del proyecto.

13. Instrucciones de despliegue

Para ejecutar el proyecto se deben seguir los siguientes pasos:

1. Clonar el repositorio GitHub del proyecto.
2. Abrir el proyecto en IntelliJ IDEA.
3. Configurar el JDK utilizado para el desarrollo.
4. Verificar que todas las dependencias del proyecto se encuentren correctamente cargadas.
5. Ejecutar la clase Main para visualizar la demostración funcional del sistema.
6. Alternativamente, ejecutar la clase SistemaHotelGUI para utilizar la interfaz gráfica incluida en el proyecto.
7. Utilizar los datos precargados para probar los distintos casos de uso implementados.

Repositorio del proyecto:

<https://github.com/131310100505/SistemaGestionHoteleria>

14. Evidencia de ejecución

Con el objetivo de validar el correcto funcionamiento del sistema, se realizaron pruebas sobre los principales casos de uso implementados.

Las evidencias presentadas demuestran el funcionamiento de las funcionalidades principales, así como la correcta integración de los patrones de diseño utilizados durante el desarrollo.

Casos de prueba ejecutados

Escenario	Resultado
Registro de huésped	Correcto
Registro de habitación	Correcto
Consulta de disponibilidad	Correcto
Creación de reserva	Correcto
Aplicación de descuentos	Correcto

Agregado de servicios adicionales	Correcto
Confirmación de reserva	Correcto
Registro de pago	Correcto
Finalización de reserva	Correcto
Validación de transición inválida	Correcto

Evidencia 1 – Registro de huéspedes

```
--- CU1: Registrar huéspedes ---  
[DB] Conexion establecida con jdbc:mysql://localhost:3306/sistema_hotel  
Huesped registrado: Huesped #1 - Ana Gomez (DNI 30111222, ana@mail.com)  
Huesped registrado: Huesped #2 - Luis Perez (DNI 28999111, luis@mail.com)
```

Evidencia 2 – Registro de habitaciones

```
--- CU2: Registrar habitaciones (Factory Method) ---  
Habitacion registrada: Habitacion 101 [STANDARD] $20000.0/noche - DISPONIBLE  
Habitacion registrada: Habitacion 201 [PREMIUM] $30000.0/noche - DISPONIBLE  
Habitacion registrada: Habitacion 301 [SUITE] $44000.0/noche - DISPONIBLE
```

Evidencia 3 – Creación de reserva

```
--- CU3: Crear reserva ---  
[EMAIL -> ana@mail.com] Su reserva #1 fue creada (PENDIENTE).  
Reserva #1 - Ana Gomez | Hab 201 | 2026-07-10 a 2026-07-13 | Estado: PENDIENTE | Total: $76500.0
```

Evidencia 4 – Confirmación de reserva

```
--- CU5: Confirmar reserva (State) ---  
-> Reserva confirmada.  
[EMAIL -> ana@mail.com] Su reserva #1 fue confirmada.  
Estado actual: CONFIRMADA
```

Evidencia 5 – Aplicación de descuentos y servicios

```
--- CU8: Calcular costo final ---  
Costo total con servicios y descuento: $79050.0
```

Evidencia 6 – Registro de pago

```
--- CU10: Registrar pago ---  
Pago registrado: Pago #1 $79050.0 (Tarjeta de credito) - 2026-06-20
```

Evidencia 7 – Historial de reservas

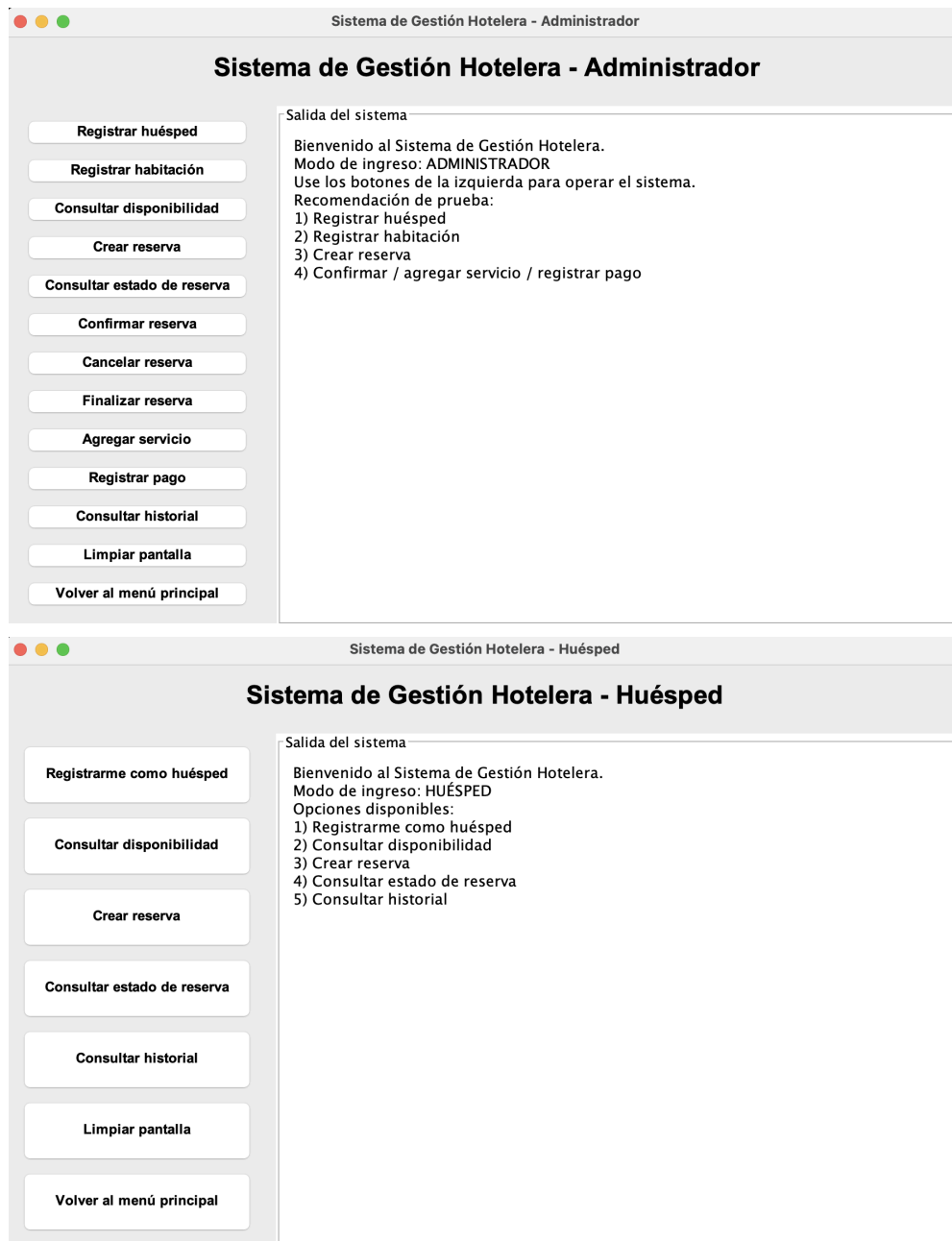
```
--- CU11: Consultar historial de reservas ---  
Reserva #1 - Ana Gomez | Hab 201 | 2026-07-10 a 2026-07-13 | Estado: FINALIZADA | Total: $79050.0  
Reserva #2 - Luis Perez | Hab 301 | 2026-08-01 a 2026-08-05 | Estado: CANCELADA | Total: $176000.0
```

Evidencia 8 – Validación de transición inválida

```
--- Validacion de transicion invalida (State) ---  
Excepcion controlada: No se puede confirmar una reserva finalizada.
```

Evidencia 9 – Interfaz gráfica del sistema





Además de la demostración por consola, el sistema incorpora una interfaz gráfica desarrollada en Java Swing que permite ejecutar los principales casos de uso mediante botones y formularios interactivos.

Esta facilita la interacción con el sistema y demuestra la correcta integración entre la capa de presentación y la lógica de negocio implementada.

15. Historial de versiones

El proyecto fue gestionado mediante GitHub, utilizando commits descriptivos para registrar la evolución del sistema durante las distintas etapas de desarrollo.

Los commits realizados reflejan tareas relacionadas con:

- Construcción del modelo de dominio.
- Implementación de controladores.
- Implementación de patrones de diseño.
- Desarrollo de repositorios y persistencia.
- Actualización de diagramas UML.
- Corrección de errores y mejoras funcionales.
- Elaboración de la documentación.

La utilización de control de versiones permitió coordinar el trabajo del equipo y mantener un registro de los cambios realizados durante el proyecto.

Merge branch 'master' of https://github.com/131310100505/SistemaGestionHotelera	6ad3b35		
tomidella committed last week			
"Cambios"	758b695		
tomidella committed last week			
"Creacion interfaz gráfica básica con Swing"	5749bc5		
tomidella committed last week			
Add files via upload	76671fe		
tomidella authored last week			
""	e8cd289		
tomidella committed last week			
actualizacion general del código	6c9fc28		
tomidella committed last week			
.	7bdffef		
tomasszk committed 2 weeks ago			
.	a6802f6		
tomasszk committed 2 weeks ago			
creacion de clases restantes	f7f571d		
tomasszk committed 2 weeks ago			
creacion facade y factory (con sus clases)	1e32ff2		
131310100505 committed 2 weeks ago			
creacion repository	c6541e7		
tomidella committed 2 weeks ago			
implementacion modelo	577660c		
tomasszk committed 2 weeks ago			

Commits on Jun 20, 2026

arreglo en interfaz y readme	aa293b4		
131310100505 committed 2 minutes ago			
arreglo en interfaz	67602fc		
131310100505 committed 11 minutes ago			
correcciones	1d34817		
131310100505 committed 2 hours ago			

Commits on Jun 18, 2026

Agrego README del proyecto	5a48e62		
tomidella committed 2 days ago			

Commits on Jun 10, 2026

Add files via upload	e1a15c5		
tomasszk authored last week			

16. Conclusión

El desarrollo del Sistema de Gestión Hotelera permitió aplicar de manera práctica los conceptos abordados durante la materia, integrando análisis, diseño e implementación dentro de una única solución de software.

La aplicación desarrollada permite gestionar huéspedes, habitaciones, reservas, promociones, servicios adicionales y pagos, cumpliendo los objetivos definidos durante las etapas iniciales del proyecto.

La utilización de patrones de diseño GoF, principios SOLID y patrones GRASP contribuyó a obtener una arquitectura organizada, extensible y de bajo acoplamiento, facilitando tanto el mantenimiento como la evolución futura del sistema.

Como principales ventajas del enfoque adoptado se destacan la reutilización de componentes, la separación de responsabilidades y la facilidad para incorporar nuevas funcionalidades sin modificar significativamente la estructura existente.

Entre las principales dificultades encontradas durante el desarrollo se destaca la necesidad de mantener coherencia entre los requerimientos, los diagramas UML y la implementación final, especialmente al incorporar distintos patrones de diseño dentro de una misma solución.

En conclusión, el proyecto permitió consolidar los conocimientos adquiridos durante la cursada y comprender la importancia del diseño orientado a objetos como herramienta fundamental para el desarrollo de software mantenible y escalable.